

SSRM Engine

SSRM Engine · docs/latest · generated 2026-08-15

The SSRM (server-side row model) engine is a transport-agnostic data service that powers server-driven grid queries, aggregations, and real-time updates. It is a pure computational component — driven through a single, well-defined interface (`ICacheIngest`) — so it can run in any host: a `SharedWorker`, a web page, `Node.js`, or any other runtime.

The hub is **one host** of this engine, not its owner. Any transport (STOMP, REST poller, Kafka bridge, in-page mock) can feed it; any query consumer can read it.

Contract surfaces

1. Ingest (transport → engine)

```
interface ICacheIngest {
  replaceSnapshot(rows: object[]): void;
  upsert(rows: object[]): void;
  remove(ids: string[]): void;
}
```

Transport implementations call these three methods to mutate the cache. The engine guarantees last-value-wins semantics: if the same row id arrives twice in one update, the last value wins. Aggregates (totals, subtotals, pivots) recompute immediately from state.

2. Query (consumer → engine)

```
interface SsrmsServer {
  getRows(request: SsrmsGetRowsRequest, sessionId?: string): SsrmsGetRowsResult;
  getSetFilterValues(req: SetFilterValuesRequest): string[];
  getStatusBar(request?: StatusBarRequest): StatusBarSummary;
  getDetailRows(request: DetailRowsRequest): Row[];
  getGrandTotal(request: Pick<SsrmsGetRowsRequest, 'filterModel' | 'valueCols' | 'quickFilterText'>): Row;
  configureExpressions(rules: ExpressionRule[], sessionId?: string): void;
  getStats(): SsrmsStats;
}
```

Consumer applications call `getRows` / `getSetFilterValues` / `getStatusBar` / `getDetailRows` / `getGrandTotal` to retrieve filtered, sorted, grouped, and aggregated results. Results are

memoized by request hash (revision-keyed) to avoid redundant computation — see Guarantees below for exactly what that memo does and does not share across sessions.

`sessionId` is an **optional trailing positional argument**, not a field on the request object (`engine.getRows(criteria, 'gridA')`). It scopes two independent things:

- **Expression rules** (`configureExpressions`) — a session with its own configured rules (even an empty array) resolves to exactly those; every other session, including a sessionless call, resolves to the rules under the internal global bucket. Two grids on the same engine with different calculated columns never see each other's.
- **Query-order memo sizing** — `SsrmServer` resizes `QueryEngine`'s order-cache via `setOrderCacheSize(max(MIN_ORDER_CACHE_SIZE, liveSessions * ORDER_CACHE_SIZE_PER_SESSION))` on every `setViewportInterest` / `clearViewportInterest` call, so the memo scales with how many live sessions (blotters) are attached instead of thrashing one static-size LRU.

The memo itself is **not** keyed by `sessionId` — see Guarantee 3.

3. Publish (engine → consumer)

```
interface SsrmServer {
  onTick(handler: (event: TickEvent) => void): Unsubscribe;
  onFlush(handler: (event: SsrmFlushEvent) => void): Unsubscribe;
}

interface SsrmFlushEvent {
  type: 'rows' | 'snapshot';
  /** Union of changed keys since the last flush (empty for snapshot). */
  keys: string[];
  /** Union of changed columns since the last flush. */
  columns: string[];
  /** Store revision AT FLUSH TIME — the consistency stamp. */
  revision: number;
  /** Keys accumulated before dedup — `keys.length` were conflated. */
  updatesAccumulated: number;
}
```

`onTick` is the raw, per-store-mutation stream (fires on every `upsert()` / `replaceSnapshot()` / `remove()` call that changes the output surface). `onFlush` is a **settled, deduped, revision-stamped** view derived from that stream — the shape a consumer should actually subscribe to:

- **publishWindowMs** (`SsrmServerOptions.publishWindowMs`, default `0`) — trailing-edge window over which `rows` ticks are accumulated and key-conflated before one `SsrmFlushEvent` is emitted. `0` (the default) is **passthrough**: one flush per store tick, matching pre-windowing per-frame behaviour. A snapshot (`replaceSnapshot`) always flushes **immediately**, even inside an open window — it invalidates whatever was accumulating.

- **Injectable timer** — `setTimer` / `clearTimer` (defaults to `setTimeout` / `clearTimeout`) let a host thread its own timer (e.g. the `SharedWorker` hub's `setInterval`-based default) through so fake-timer tests control exactly when a window fires, with no reliance on wall-clock waits.
- **Cache ingest is never throttled** — only the *notification* is windowed.
`upsert` / `replaceSnapshot` / `remove` apply to the store synchronously and immediately; a `getRows()` call mid-window always reads current state. Windowing only delays when listeners are *told* something changed.
- **revision is stamped at flush time**, not at accumulation start — so a listener reacting to a flush is guaranteed a store state at least as fresh as that revision, never a stale one from earlier in the window.
- **`dispose()`** unsubscribes from the raw tick stream and clears any pending window timer — ingest after `dispose()` still succeeds, it just produces no further flush and arms no new timer.

See Guarantees below for the consistency property this buys: a value that spikes and retreats within one window is never published, only the settled final value, identically to every subscriber.

Recipe: any transport drives the engine

A transport is any object that calls `ICacheIngest`. Here is a complete example using an in-memory fake — the pattern adapts to HTTP, message queues, or WebSocket listeners:

```
/** Any transport is just something that calls ICacheIngest. */
function fakeTransport(sink: ICacheIngest) {
  return {
    snapshot: (n: number) =>
      sink.replaceSnapshot(
        Array.from({ length: n }, (_, i) => ({
          id: `P${i}`, book: i % 2 ? 'A' : 'B', px: i,
        })),
      ),
    tick: (id: string, px: number) => sink.upsert([ { id, px } ]),
    drop: (id: string) => sink.remove([id]),
  };
}

// Use it:
const engine = new SsrServer({ keyColumn: 'id' });
const transport = fakeTransport(engine);

transport.snapshot(500); // engine sees 500 rows
transport.tick('P7', 9_999); // engine updates row P7, recomputes aggregates
const result = engine.getRows({ startRow: 0, endRow: 100 }); // query it
```

Guarantees

1. **Last-value-wins cache:** if the same row id arrives multiple times in a single update, the final state reflects only the last value. Partial updates (upserts) merge with existing rows.
2. **Recompute-from-state aggregates:** every query reads the current cache and recomputes totals, subtotals, pivots, and grand totals. No separate aggregate store; the cache is the source of truth.
3. **Revision-keyed memoization, shared across sessions:** results are cached by a hash of the query request (filter/sort/group/pivot/value-cols shape — `sessionId` is **not** part of the key). Cache invalidates on mutation; identical queries within the same revision, from *any* session, reuse the prior result. The grand-total aggregate specifically is memoized **pre-enrichment**, so two sessions issuing the identical `getRows` criteria at one revision get back the exact same `grandTotalData` object — not merely an equal value, the same reference (`toBe` , not `toEqual`). Enrichment (calculated/expression columns, which are per-session — see Contract 2) always runs on the sliced page *after* memo retrieval, so sharing the memo across sessions with different expression rules is safe.
4. **Windowed-flush consistency:** with `publishWindowMs > 0` , any value that changes and then changes back within one window is never observed by a flush listener — only the value settled at the moment the window fires is published, and every listener (every grid/session) sees that same settled value at that same `revision` . A window never delays or throttles what `getRows()` itself returns; it only delays the *notification* that a change happened. See `engineContract.test.ts` 's `cross-grid consistency acceptance` suite for the acceptance form of both this and Guarantee 3.
5. **Observability:** `SsrmServer.getStats(): SsrmStats` (additive over the store's own `CacheStats`) reports `sessions` (live viewport-interest sessions), `memoHits` / `memoMisses` (cumulative order-memo), and flush counters `flushes` / `updatesAccumulated` (pre-dedup key updates) / `keysFlushed` (post-dedup keys shipped) — `updatesAccumulated` / `keysFlushed` is the conflation ratio a window bought. `npm run bench:ssrm` 's "Windowed publish" section reports this side by side for a passthrough vs. windowed engine over the same tick load.

Standalone import

The engine is published as its own subpath, with no SharedWorker/hub/RPC dependency:

```
import { SsrmServer, RowStore, QueryEngine } from '@wellsfargo-starui/data/ssrm-engine';
import type { SsrmFlushEvent, SsrmStats, ICacheIngest } from '@wellsfargo-starui/data/ssrm-engine';
```

This is the same entry point `engineContract.test.ts` and `scripts/bench-ssrm.mjs` import against (the benchmark loads the **built** `host-data/dist/runtime/ssrm/index.js` , so run `npm run build:packages` before `npm run bench:ssrm`).